

Partial Assembly by L^2 -Projection: node-level performance of generated inexact tangent kernels for finite-strain elasticity

Patrick Zulian*

September 7, 2026

Abstract

The tangent operator of a finite-strain elasticity problem is applied tens to hundreds of times per Newton step, and every application re-evaluates the same constitutive tangent at every quadrature point of every element, because the tangent varies within the element and the increment is all that changes between Krylov iterations. We eliminate that redundancy by replacing the pulled-back material tangent with its element-wise L^2 projection onto the constants. The projection factors the integrand into a per-element tensor, assembled once per Newton step, and a purely geometric tensor belonging to the reference element, which is integrated exactly and symbolically at code-generation time and folded into the emitted kernel as literal constants. The quadrature loop leaves the application entirely: it reads a 45-number store and the increment, and touches neither geometry, state, nor material parameters.

We give a per-element cost model — floating-point operations and compulsory memory traffic — that places the resulting kernels between 1.0 and 6.9 flop/byte of arithmetic intensity depending on element type, and predicts that linear tetrahedra are limited by the store bandwidth while quadratic tetrahedra and trilinear hexahedra are limited by the contraction. The generator exploits the second regime by selecting between two contraction schedules per element on symbolic operation count, which reduces the trilinear hexahedral kernel from 2732 to 1801 operations.

On 1.04×10^7 degrees of freedom the generated kernels reach 219.6 MDOF/s on quadratic tetrahedra against 137.4 for a hand-optimised partial-assembly implementation of the same neo-Hookean material, with the assembly phase also $1.67\times$ faster, and reach parity on trilinear hexahedra. Measured against the exact matrix-free application the speed-ups are $8.1\times$, $2.8\times$ and $2.4\times$ for the three element types, with break-even below 1.2 applications per assembly. Strong scaling on 72 Arm Neoverse-V2 cores is $69.4\times$ for the exact application and $73.5\times$ for the stored one. The approximation is exact on affine simplices and converges at $O(h^{1.07})$ elsewhere.

Contents

1	Introduction	2
2	Background and related work	3
3	Method	4
3.1	Formulation	4
3.2	Accuracy	4
3.3	What is stored, and when symmetry may be assumed	5

*Affiliation to be completed.

4	Structure of the reference tensor	5
5	Kernel synthesis	6
5.1	Two contraction schedules	6
5.2	Constant folding versus runtime tabulation	7
5.3	Vectorisation and register pressure	7
6	Cost model	8
7	Implementation	9
8	Experimental setup	9
9	Results	10
9.1	Validation	10
9.2	Accuracy of the approximation	10
9.3	Node-level throughput	11
9.4	Achieved rates against the model	11
9.5	Strong scaling	12
9.6	Store precision	12
10	Discussion	13
11	Conclusions	14

1 Introduction

Newton–Krylov solvers for finite-strain elasticity have a lopsided cost profile. A Newton step forms one residual and then applies the tangent operator n_{it} times inside a Krylov method, where n_{it} is typically in the tens to the hundreds. In a matrix-free implementation each of those applications evaluates the constitutive tangent at every quadrature point of every element. All n_{it} evaluations produce identical results: the state is frozen within the step and only the increment changes.

The redundancy is well known and the standard responses are sparse assembly, which pays once but produces an object that is expensive in memory and bandwidth-bound to apply, and *partial assembly*, which stores a compact per-element quantity capturing geometry and state and applies it by a small tensor contraction. Partial assembly is routine in high-order matrix-free codes for linear operators [20, 10, 16, 15, 7]. It does not transfer directly to a nonlinear material, because the object one would store — the pulled-back tangent $\bar{S}$ — is not constant over the element.

This paper takes the obvious step and measures the consequences. We replace $\bar{S}$ by its element average, which factors the integrand into a per-element store and a reference-element tensor. The interesting content is not the approximation, which is elementary, but what becomes possible once it is made: the reference tensor can be integrated exactly in rational arithmetic before compilation, its rank structure can be exposed and exploited symbolically, and the resulting element kernel contains no loads other than the store and the increment, no branches, and no constants that are not compile-time immediates. The questions this raises are node-level performance questions: what is the arithmetic intensity of the resulting kernel, which bound is it against, how does it vectorise, how does it scale, and does it beat a hand-optimised kernel for the same material.

Contributions

1. A partial-assembly formulation for nonlinear elasticity by L^2 projection of the pulled-back tangent (Section 3), whose application reads only a 45-number per-element store and the increment.
2. A structural analysis of the reference tensor (Section 4): it is a Gram matrix, so its rank is the dimension of the span of the element’s gradient functions, which yields an exact rational factorisation and a four-stage contraction schedule.
3. A second contraction schedule and a per-element selection rule on symbolic operation count after common-subexpression elimination (Section 5), reducing the trilinear hexahedral kernel by 34% and raising its throughput from 100 to 132 MDOF/s.
4. A per-element cost model in flops and compulsory bytes (Section 6), giving arithmetic intensities of 1.0, 4.9 and 6.9 flop/byte for TET4, TET10 and HEX8, and predicting distinct bottlenecks for the three.
5. A node-level performance study (Section 9) at 1.04×10^7 degrees of freedom against hand-optimised kernels for the same materials, with achieved flop and bandwidth rates derived from the model, strong scaling to 72 cores, and a store-precision study.
6. A practical result on storage: the symmetry that halves the store holds for energy-based materials and fails for residual-based ones, and assuming it for Kelvin–Voigt viscoelasticity produces a different operator with 5.3×10^{-2} relative error (Section 3.3).

2 Background and related work

Matrix-free operator application. Storing per-quadrature-point geometric data and applying an operator by tensor contraction originates in spectral-element methods [20, 10] and underpins modern matrix-free frameworks: the cell-based interface of `deal.II` [16, 17], MFEM [2], and libCEED [7], whose “Q-data” plays the role of $\bar{S}$ here. The performance of these frameworks rests on sum factorisation, whose asymptotic advantage appears at polynomial degrees well above those considered in this work [18, 15]. Our regime is complementary: low to moderate order, where element tensors are small enough to be manipulated exactly and symbolically, and where the dominant cost is the repeated evaluation of a nonlinear constitutive law rather than basis-function contraction.

Nonlinear matrix-free solvers. Brown [6] analyses matrix-free Newton–Krylov methods for nodal high-order elements in three dimensions and identifies tangent evaluation as the bottleneck. Jacobian-free Newton–Krylov methods [14] avoid forming the tangent at all at the price of a directional-difference approximation; we instead form an approximate tangent explicitly, which is cheaper to apply repeatedly and whose error is a property of the discretisation rather than of a difference parameter. The convergence theory for solvers with inexact Jacobian information is classical [9, 11].

Code generation. Automatic generation of element kernels from a symbolic weak form is established [13, 3, 21]. What distinguishes the generator described here is that the projection, the exact rational integration, the rank factorisation and the schedule selection are all performed symbolically, and the reference tensor appears in the emitted code only as folded literal constants — never as a runtime table.

Reduced precision. Storing the assembled tensor at reduced precision connects to the mixed-precision literature [12, 1]. Our finding (Section 9.6) is negative and instructive: **fp16** does not pay on either machine tested, because the kernels are not at the bandwidth bound at the problem sizes where the store would dominate.

3 Method

3.1 Formulation

Let K be an element with reference cell $\hat{K}$, map $\mathbf{x}(\boldsymbol{\xi})$, Jacobian J and $dV = \det J d\boldsymbol{\xi}$, and let $\{\varphi_p\}_{p=1}^N$ be the reference basis. For a hyperelastic material with first Piola–Kirchhoff stress $P(F)$ [19, 4] the matrix-free tangent action on an increment h is

$$(\mathcal{H}h)_{i,p} = \int_K S_{ijkl}(F(\mathbf{x})) \frac{\partial h_k}{\partial x_l} \frac{\partial \varphi_p}{\partial x_j} dV, \quad S_{ijkl} = \frac{\partial P_{ij}}{\partial F_{kl}}. \quad (1)$$

Pulling the derivatives back with $\partial/\partial x_j = \sum_n J_{nj}^{-1} \partial/\partial \xi_n$ and absorbing the geometry into the tangent,

$$\bar{S}_{ikmn} = \sum_{j,l} S_{ijkl} J_{nj}^{-1} J_{ml}^{-1} \det J, \quad (\mathcal{H}h)_{i,p} = \int_{\hat{K}} \bar{S}_{ikmn}(\boldsymbol{\xi}) \frac{\partial h_k}{\partial \xi_m} \frac{\partial \varphi_p}{\partial \xi_n} d\boldsymbol{\xi}. \quad (2)$$

Everything under this integral except $\bar{S}$ belongs to the reference element. The quadrature loop exists solely because $\bar{S}$ depends on $\boldsymbol{\xi}$, through F .

Replacing $\bar{S}$ by its element average $\bar{S}^0 = |\hat{K}|^{-1} \int_{\hat{K}} \bar{S}$ — its L^2 projection onto the constants — removes it from the integral:

$$(\mathcal{H}h)_{i,p} \approx \sum_{k,m,n} \bar{S}_{ikmn}^0 \sum_j h_{k,j} \bar{W}_{jmpn}, \quad \bar{W}_{jmpn} = \int_{\hat{K}} \frac{\partial \varphi_j}{\partial \xi_m} \frac{\partial \varphi_p}{\partial \xi_n} d\boldsymbol{\xi}. \quad (3)$$

The two factors have entirely different lifetimes. $\bar{S}^0$ is a small per-element tensor computed once per Newton step; $\bar{W}$ depends only on the element type and is integrated exactly, in rational arithmetic, before compilation. Algorithm 1 states the resulting split.

Algorithm 1 Partial assembly by projection, within one Newton step

- 1: **once per Newton step:**
 - 2: **for all** elements K **in parallel do**
 - 3: evaluate F , $S(F)$ and J on K ; form $\bar{S}$ by (2)
 - 4: $\bar{S}_K^0 \leftarrow$ element average of $\bar{S}$ ▷ 45 or 81 numbers
 - 5: **end for**

 - 6: **once per Krylov iteration:**
 - 7: **for all** elements K **in parallel do**
 - 8: gather $h|_K$; contract with $\bar{S}_K^0$ and the folded constants of $\bar{W}$
 - 9: scatter the result ▷ no geometry, state or material read
 - 10: **end for**
-

3.2 Accuracy

On an affinely mapped simplex with a P1 basis, J and ∇h are constant, hence F and $\bar{S}$ are constant, hence $\bar{S}^0 = \bar{S}$ and (3) is an identity: the projection is exact for any constitutive law,

nonlinear or not. This is elementary but operationally central, because it gives a configuration in which the approximate kernel must reproduce the exact one to round-off, independently of every other component of the implementation. It is the control for every measurement in Section 9.

Elsewhere the projection reproduces constants and is therefore first-order accurate in the element size by the standard Bramble–Hilbert argument [5, 8]. The measured rate is $O(h^{1.07})$ (Section 9.2).

3.3 What is stored, and when symmetry may be assumed

The element matrix implied by (3) is $\bar{S}_{ikmn}^0 \bar{W}_{jmpn}$, and since $\bar{W}_{jmpn} = \bar{W}_{pnjm}$, requiring it to be symmetric under exchange of the two (component, node) pairs is equivalent to

$$\bar{S}_{ikmn}^0 = \bar{S}_{kinm}^0, \quad (4)$$

which reduces the store from 81 to 45 numbers in three dimensions and from 16 to 10 in two. The pair that exchanges is (i, n) against (k, m) : the output component with the *test* direction, and the increment component with the *increment* direction. Packing on (i, k) against (m, n) instead is a consistent error, invisible to any test that compares one form of the construction against another, and detectable only by comparison against (1) integrated independently.

Whether (4) holds at all is a property of the formulation. A material given by a strain energy has a flux that is a gradient, so its tangent is a Hessian and the symmetry follows. A material given directly by a residual has no such guarantee: its tangent is a Jacobian and need not be symmetric. Kelvin–Voigt viscoelasticity is a case in point; for the parameters used in Section 9 we measure $\max_{ijkl} |A_{ijkl} - A_{klij}| = 0.25$, so folding it into 45 components averages the antisymmetric part away and produces a *different operator*, with 5.3×10^{-2} relative error on TET4 — an element on which the projection itself is exact. The generator therefore decides symmetry per material, symbolically, and defaults to unsymmetric: storing a full square for a symmetric tangent wastes bandwidth, whereas storing half of an unsymmetric one is silently wrong.

4 Structure of the reference tensor

Because $\bar{W}$ is integrated exactly its entries are rationals, and there are very few distinct ones: at most ten for any element considered (Table 1). On simplices most entries vanish.

element	entries	distinct values	zeros	projection
TRI3	36	3	20	exact
TET4	144	3	108	exact
QUAD4	64	6	0	inexact
HEX8	576	10	0	inexact
TET10	900	9	519	inexact

Table 1: The symbolically integrated reference tensor $\bar{W}$.

The sparsity is a symptom of a stronger property. Read as a matrix on the pair index (j, m) , $\bar{W}$ is by definition the Gram matrix in $L^2(\hat{K})$ of the family $\{\partial\varphi_j/\partial\xi_m\}$, so its rank is the dimension of the span of that family (Table 2). A linear simplex is rank one because its gradients are constants, which is precisely the classical geometric-factor formulation: one object per element, contracted once. The rank structure is therefore not a separate mechanism but the same one carried to elements whose gradients vary.

element	order Nd	rank r	span of the gradient functions
TRI3	6	1	constants
TET4	12	1	constants
QUAD4	8	3	1, ξ_0 , ξ_1
HEX8	24	7	1 and the six bilinear monomials
TET10	30	4	1, ξ_0 , ξ_1 , ξ_2

Table 2: Rank of the reference tensor, which equals the dimension of the span of the element’s gradient functions.

Since the rank is known and the entries rational, the factorisation

$$\overline{W} = C^\top X C, \quad C \in \mathbb{Q}^{r \times Nd}, \quad X \in \mathbb{Q}^{r \times r}, \quad (5)$$

is computed exactly, with no tolerance and no loss, and verified against the tensor it factors over the rationals.

5 Kernel synthesis

5.1 Two contraction schedules

Substituting (5) into (3) and naming each intermediate gives a four-stage schedule:

$$P_{kam} = \sum_j C_{a,(j,m)} h_{k,j} \quad \text{compress the increment} \quad (6)$$

$$Y_{ian} = \sum_{k,m} \overline{S}_{ikmn}^0 P_{kam} \quad \text{apply the tangent} \quad (7)$$

$$Q_{ibn} = \sum_a X_{ab} Y_{ian} \quad \text{apply the middle factor} \quad (8)$$

$$\text{out}_{ip} = \sum_{b,n} C_{b,(p,n)} Q_{ibn} \quad \text{expand back to nodes} \quad (9)$$

Every sum skips its zero coefficients, so a vanishing entry never enters an expression rather than being cancelled out of one afterwards. The staging is load-bearing: expanding (6)–(9) into one expression per output and letting common-subexpression elimination rediscover the structure recovers the dense operation count exactly.

Staging is nonetheless not always cheaper. It must compress the increment and expand the result, and where the rank is a large fraction of the order those two stages cost more than the factorisation saves. The alternative contracts $\overline{W}$ against the increment directly,

$$G_{kmpn} = \sum_j \overline{W}_{jmpn} h_{k,j}, \quad \text{out}_{ip} = \sum_{k,m,n} \overline{S}_{ikmn}^0 G_{kmpn}, \quad (10)$$

so that $\overline{S}^0$ appears once, in a single 27-term contraction per output, with nothing to compress or expand.

Which schedule wins is a property of the element, so the generator constructs both symbolically and emits the cheaper (Table 3). On HEX8 the selection reduces the emitted application from 2732 to 1801 operations, against 1558 for a hand-written kernel for the same contraction, and raises measured throughput from 100 to 132 MDOF/s (Section 9.3).

The comparison must be made *after* elimination. On the unsimplified expressions the direct schedule appears inferior on every element, because elimination is exactly what removes its redundancy; a selection rule using the pre-elimination count rejects it universally.

element	staged	direct	r of Nd	selected
TET4	207	342	1 of 12	staged
TET10	1380	1605	4 of 30	staged
HEX8	2610	1731	7 of 24	direct

Table 3: Floating-point operations per element for the two schedules, counted after common-subexpression elimination. The factorisation ceases to pay once the rank reaches a sufficient fraction of the order.

5.2 Constant folding versus runtime tabulation

The entries of $\overline{W}$, C and X are known at generation time and are emitted as literals. Tabulating the distinct values at runtime and indexing them — the apparently attractive alternative — compresses a negligible amount of memory and destroys the arithmetic: the constants cease to be constants, the compiler can no longer fold them, and every multiplication by a zero entry survives into the emitted code. A tabulated TET4 kernel stores a table containing exactly 1, -1 and 0, and then multiplies by the zero entry twelve times per kernel; the nominal “97.9% memory saving” buys 141 stored numbers that were 0 and ± 1 at a cost of several hundred operations (Table 4).

element	r/Nd	dense, folded	staged, folded	runtime table
TRI3	1/6	96	52	244
TET4	1/12	486	207	1467
QUAD4	3/8	238	238	431
HEX8	7/24	4059	2610	6330
TET10	4/30	4018	1380	8349

Table 4: Floating-point operations per element application after common-subexpression elimination. Symbolic counts, not measured run times.

5.3 Vectorisation and register pressure

Kernels process a block of elements at a time with the lane loop innermost. Three properties of the staged form matter for that layout.

The reference tensor is lane-invariant. C and X are compile-time rationals, identical for every lane. Folded, they appear as immediates: no load, no broadcast, and multiplication by ± 1 or a power of two is eliminated. Behind a runtime table the same values become a lane-independent memory reference — a broadcast load rather than an immediate — and, more damagingly, an opaque value that no longer permits folding.

The store is a unit-stride stream. $\overline{S}^0$ is held one component-array per index in a structure-of-arrays layout, so each of the 45 components is a unit-stride read across the lane loop and vectorises without gather.

The intermediates are lane-width vectors. P , Y and Q are per-element scalars, so each occupies one vector register; all three stage sizes are d^2r , so rank governs register pressure (Table 5). On HEX8, $63 + 63 + 63$ live intermediates at a lane width of eight will not stay in registers and the kernel spills.

The two difficulties therefore coincide: on the elements where the projection is exact the register pressure is lowest, and on HEX8, where it is an approximation, register pressure is

element	r	$ P $	$ Y $	$ Q $	outputs
TRI3	1	4	4	4	6
TET4	1	9	9	9	12
QUAD4	3	12	12	12	8
HEX8	7	63	63	63	24
TET10	4	36	36	36	30

Table 5: Live intermediates per element in each stage; in a blocked kernel each is one vector register.

highest — and HEX8 is also the element on which the staged schedule loses to the direct one, which needs no intermediates. Rank truncation, keeping $r' < r$ modes, remains available for elements where staging still wins, at the cost of a second and larger approximation.

The final stage writes one value per node per component and reuses the conventional scatter: atomic accumulation on an unordered traversal, plain accumulation into thread-private scratch on a block-ordered one.

6 Cost model

We model the stored application per element. Let n_c be the number of stored components, s_t the store precision in bytes, N the node count, $d = 3$, and s_i , s_r the index and vector precisions. Compulsory traffic per element is

$$B_e = \underbrace{n_c s_t}_{\text{store}} + \underbrace{N s_i}_{\text{connectivity}} + \underbrace{\frac{2Nd s_r}{\kappa}}_{\text{increment and output}}, \quad \kappa = \frac{N n_{\text{elem}}}{n_{\text{nodes}}}, \quad (11)$$

where κ is the mean number of elements sharing a node, which amortises the gathered increment and the scattered output over the elements that reuse them. The store is streamed once and never reused, so it is charged in full.

For the configuration measured here, $s_t = 4$ (fp32 store), $s_i = 4$ (32-bit indices), $s_r = 8$ (fp64 vectors), $n_c = 45$; and for the structured meshes used, $\kappa = 24, 7.5$ and 8 for TET4, TET10 and HEX8 respectively. Taking W_e from the *selected* schedule (Table 3) gives the intensities in Table 6.

element	N	κ	W_e (flop)	store (B)	other (B)	B_e (B)	$I_e = W_e/B_e$ (flop/B)
TET4	4	24	207	180	24	204	1.01
TET10	10	7.5	1380	180	104	284	4.86
HEX8	8	8	1801	180	80	260	6.93

Table 6: Per-element cost of the stored application from (11). The store is 88% of TET4’s compulsory traffic and 63% of TET10’s.

The model separates the three elements sharply. TET4 sits at 1.0 flop/byte: it does so little arithmetic that its cost is essentially the cost of streaming its own store, and it is the element for which reduced store precision should matter most. TET10 and HEX8, at 4.9 and 6.9 flop/byte, are on the compute side of any plausible machine balance, and their performance should be governed by the contraction — which is what makes the schedule selection of Section 5 worth performing.

Converting to per-degree-of-freedom quantities requires the element-to-dof ratio $\rho = n_{\text{elem}}/n_{\text{dof}}$, which for the structured meshes used is 2, 1/4 and 1/3 for TET4, TET10 and HEX8. Table 7 gives the resulting figures, which are what the measured throughputs of Section 9.3 must be multiplied by to obtain achieved rates.

element	ρ	flop/dof	byte/dof
TET4	2	414	408
TET10	1/4	345	71
HEX8	1/3	600	87

Table 7: Per-degree-of-freedom cost of the stored application. TET4 moves nearly six times the bytes per degree of freedom of the other two, because a structured tetrahedral mesh has six elements per lattice cell and each carries its own store.

A consequence worth stating in advance of the measurements: partial assembly is not uniformly attractive. Its benefit is the arithmetic it removes, and TET4 has little to remove; its store, by contrast, is the largest per degree of freedom of the three. The method should therefore help TET4 least and TET10 most, and Section 9.3 confirms this ($2.4\times$ against $8.1\times$ over the exact application).

7 Implementation

The scheme is implemented in the code generator of SFEM, an open-source finite-element library. From a symbolic material description the generator derives the flux, differentiates it, performs the pullback, decides symmetry, integrates $\overline{W}$ exactly, computes the factorisation (5), selects the schedule by operation count, and emits per element type an *assembly* kernel consuming geometry and state and producing the store, and an *application* kernel consuming the store and the increment. A third kernel reads a reduced-precision store with a per-element scaling factor applied to the outputs rather than to the components.

The kernels reach the library’s solver stack through three operator methods whose defaults decline, so existing operators and callers are unaffected: a query reporting whether the approximate path is available for the current mesh and element type, an explicit update assembling the store from the current state, and an application taking the increment alone. Making the update explicit is deliberate — the store remains valid until the next update, which is the reuse the method exists to exploit, and the application takes no state argument, so it cannot silently consume a stale one.

8 Experimental setup

Machines. **M1:** Apple M1 Max, 8 performance and 2 efficiency cores, Homebrew Clang 20.1.8, OpenMP enabled. Nominal double-precision vector peak across the eight performance cores is approximately 410 GFLOP/s (four 128-bit NEON FMA pipes per core at ~ 3.2 GHz); this figure is derived from the microarchitecture, not measured. Nominal package memory bandwidth is 400 GB/s, of which the CPU complex can sustain roughly half. **Grace:** one node of a GH200 system (CSCS Alps) with 72 Arm Neoverse-V2 cores.

Build. `real_t` is `double`, `geom_t` and `metric_tensor_t` are `float`, `idx_t` is `int32_t`. All operators within a comparison are built identically and run in the same process; figures from different machines or configurations are never combined in one table.

Timed region. The timed region contains the operator application and nothing else; output initialisation, allocation and correctness checking are outside it, and each sample averages many repetitions so that no measurement is charged OpenMP team creation or a cold cache. This is not pedantry: a serial output initialisation inside the timed region perturbs the single-thread figure by 6% (2.52 against 2.68 MDOF/s) and the eight-thread figure by 44% (13.04 against 18.75), reporting a speed-up of $5.4\times$ where the true figure is $7.0\times$. The distortion grows with thread count, which is the worst possible shape for a scaling study.

Problem sizes. Throughput is reported only at sizes that saturate the machine. The comparison against library operators uses 1.04×10^7 dof; the scaling study uses 2.07×10^5 dof. Every table states its size, thread count and machine.

Comparison frame. Calling a kernel directly with precomputed geometry and calling it through the library’s operator interface are not interchangeable frames: at 1.70×10^5 dof the same kernel measures approximately 5 MDOF/s in the first and 1.46 in the second. Only the second is comparable with library operators, and all figures below are taken in it.

Accuracy measurements. Taken at non-power-of-two resolutions. On a mesh whose spacing is a power of two the entries of an `fp32` store are exactly representable and the difference from the exact application collapses to 10^{-14} , suggesting a far stronger conclusion than the truth (Table 13). Convergence rates are measured against a white-noise increment, for the reason given in Section 9.2.

9 Results

9.1 Validation

The construction is verified at four independent points: the symbolic integration against four-point Gauss–Legendre quadrature on QUAD4 and HEX8, entry by entry, to 10^{-16} ; the factorisation against the tensor it factors, exactly over the rationals; the action against (1) integrated independently on a concrete tetrahedron, for linear elasticity and neo-Hookean Ogden material; and the generated operator inside the library, driven as a solver drives it.

Only the third detects an incorrect index convention, and it did: the symmetry (4) was initially implemented as (i, k) against (m, n) , which the first two checks and any comparison of the staged schedule against the dense form all pass, because they share the error. This is the practical reason for stating the index pairing explicitly in Section 3.3.

9.2 Accuracy of the approximation

On affine simplices the approximation is exact and the measured error is at round-off, uniformly across a sweep of increasing geometric distortion. Elsewhere it is measured by warping the reference configuration by an amount w , verifying $\det F > 0$ at each step, and comparing against the exact application (Table 8).

TET10 is two orders of magnitude worse than HEX8 at comparable distortion. Both are quadratic in the geometry, but TET10 carries a quadratic *displacement* whose gradient varies linearly over the cell, and that variation is exactly what projection onto the constants discards.

Measuring the convergence rate requires care in the choice of increment. Against a smooth increment the TET10 error remains at 1.0×10^{-2} across an eightfold refinement and appears not to converge. The cause is in the metric, not the operator: for a smooth field the assembled result is a discrete second derivative, so contributions from neighbouring elements cancel and the denominator of the relative error collapses by an additional order, while the per-element

warp w	0	0.4	1.6
TET4	1.0e−15	1.0e−15	1.0e−15
TET10	2.4e−14	1.0e−02	4.3e−02
HEX8	2.7e−15	—	3.7e−04

Table 8: Relative error of the approximate application against the exact one. Neo-Hookean Ogden, $\approx 2.0 \times 10^5$ dof, M1, one thread. The $w = 0$ column is the control: it recovers the exactness of the affine case and confirms the sweep measures geometry rather than the harness.

projection errors carry independent signs and do not cancel. Against a white-noise increment the same kernel converges at $O(h^{1.07})$. At $w = 0.4$ the measured sequences are TET10 smooth $1.07\text{e}−2 \rightarrow 9.93\text{e}−3$ (flat) against random $2.73\text{e}−3 \rightarrow 2.95\text{e}−4$, and HEX8 smooth $1.16\text{e}−3 \rightarrow 2.08\text{e}−5$.

9.3 Node-level throughput

element	hand-optimised PA	exact MF	this work	vs. PA	vs. MF
TET10	137.4	27.2	219.6	1.60×	8.07×
HEX8	122.5 ± 7%	47.5	132	parity	2.78×
TET4	—	47.9	116.9	—	2.44×

Table 9: Application throughput in MDOF/s. Neo-Hookean Ogden, 1.04×10^7 dof, M1, eight threads, identical build for all columns. “PA” is a hand-optimised partial-assembly implementation of the same material, available for TET10 and HEX8 but not TET4; “MF” is the exact matrix-free application. HEX8 entries are five-run means; the run-to-run spread of the hand-optimised operator on that element is $\pm 7\%$, so the comparison there is a statistical tie. Before the schedule selection of Section 5 the HEX8 figure for this work was 100.2.

Assembly, timed separately at the same problem size, is 0.392s for the generated store against 0.653s for the hand-optimised implementation on TET10, and 0.414s against 0.427s on HEX8. On TET10 the method is therefore ahead on both phases; on HEX8 the assemblies are level, so the entire remaining difference is in the contraction — consistent with Table 3, where the emitted HEX8 kernel is 1801 operations against a hand-written 1558.

The ordering of the speed-ups over the exact application — 8.07× on TET10, 2.78× on HEX8, 2.44× on TET4 — is the ordering the cost model predicts in Section 6, and for the reason given there: the method removes arithmetic, and TET4 has the least to remove and the largest store per degree of freedom.

Break-even. The number of applications per assembly at which the method pays requires a reference performing no setup of its own, which excludes the hand-optimised operator; the exact matrix-free application is the appropriate denominator. Against it, TET10 breaks even at 1.03 applications per assembled tangent and TET4 at 1.16. A Krylov method performs tens to hundreds of applications per Newton step, so the assembly is amortised after the first iteration.

9.4 Achieved rates against the model

Multiplying the measured throughputs of Table 9 by the per-degree-of-freedom costs of Table 7 gives the achieved rates in Table 10.

The picture is consistent with the model. TET10 and HEX8 reach 18–19% of nominal vector peak while moving under 16 GB/s, an order of magnitude below what the memory

element	MDOF/s	GFLOP/s	% of peak	GB/s	I_e
TET4	116.9	48.4	12%	47.7	1.01
TET10	219.6	75.8	18%	15.6	4.86
HEX8	132.0	79.2	19%	11.4	6.93

Table 10: Achieved rates derived from Tables 7 and 9. M1, eight threads, 1.04×10^7 dof. “% of peak” is against the nominal 410 GFLOP/s double-precision vector peak of the eight performance cores.

system sustains: they are compute-bound, and the schedule selection is the right lever. TET4 reaches only 12% of peak while moving 47.7 GB/s, roughly a quarter of the sustainable CPU-side bandwidth, at an intensity of 1.01 flop/byte — close enough to the machine balance that neither bound dominates cleanly, which is the least favourable position to be in and matches its being the element that benefits least.

That 18–19% of vector peak is the ceiling for the compute-bound elements deserves comment. The kernels contain no branches and no gathers in the contraction itself, but they retain a gather of the increment and a scatter of the output at element boundaries, and the intermediates of Table 5 exceed the register file on HEX8. Closing that gap is the principal remaining optimisation and is discussed in Section 10.

9.5 Strong scaling

machine	variant	1	8	32	64	72	speed-up
M1	exact MF	2.68	18.75	—	—	—	$7.0\times (8)$
M1	stored, fp16	7.61	55.15	—	—	—	$7.2\times (8)$
Grace	exact MF	1.33	10.68	41.65	82.13	92.28	$69.4\times$
Grace	stored, fp16	2.85	25.30	97.67	191.62	209.60	$73.5\times$

Table 11: Throughput in MDOF/s against thread count. Mooney–Rivlin with Kelvin–Voigt viscosity, 2.07×10^5 dof. Parenthesised thread counts give the point at which the M1 speed-up is measured.

Grace scales essentially linearly to the full node: 96% parallel efficiency for the exact application, and above unity for the stored one, the latter reflecting improved cache residency as the per-thread working set shrinks. The atomic scatter, which is the obvious candidate for a scalability limit, is not one at this core count.

On M1 every variant peaks at eight threads and falls by approximately a fifth at ten. This is heterogeneity, not contention: the M1 Max has eight performance and two efficiency cores, a static loop schedule assigns them equal chunks, and the loop waits for the slower ones. An earlier reading of the same data attributed the plateau to the scatter; the Grace measurements refute it, and we record the episode as a caution against inferring an algorithmic limit from a single heterogeneous machine.

9.6 Store precision

Table 12 gives the footprints. For a symmetric material in three dimensions the store is 45 numbers per element; for the mixed Mooney–Rivlin/Kelvin–Voigt material, which contributes one symmetric and one unsymmetric part, it is $45 + 81 = 126$.

Table 13 isolates representation error from projection error by measuring on configurations where the projection is exact, so the whole discrepancy is the store.

components per element	fp64	fp32	fp16
45 (symmetric)	360	180	90
126 (symmetric + unsymmetric)	1008	504	260

Table 12: Bytes of stored tangent per element. The **fp16** row for 126 components includes a per-element **fp32** scaling factor, which is why it exceeds half the **fp32** figure.

element	$n = 8$	$n = 12$	$n = 16$
TET4	6.1e−16	1.2e−07	1.8e−15
TET10	1.4e−14	1.0e−05	4.8e−14
HEX8	—	6.8e−07	—

Table 13: Relative difference between the **fp32**-store application and the exact one at three mesh resolutions. The power-of-two columns are artefacts of exact representability; $n = 12$ is the honest figure.

The application therefore reproduces the exact one to the precision of its store, and that precision is element-dependent: the quadratic basis of TET10 gives its tangent a wider dynamic range and costs roughly two decimal digits more than TET4 for the same **fp32** store.

Halving the store again does not pay. At one thread the **fp16** store is slower than **fp32** on both machines, and at 72 threads it is within two per cent while costing four decimal digits. The cost model explains why: at 2.07×10^5 dof the working set is small enough to remain resident, so the kernel is not at the bandwidth bound where a smaller store would help, and the conversion cost is not recovered. The model does predict that **fp16** should help TET4 at large problem sizes, where its 408 byte/dof store dominates; that measurement has not been taken and is the most direct test of the model this work leaves open. We recommend **fp32** as the operating point.

10 Discussion

Where the method pays. It converts state-dependent work into a one-off assembly plus a state-independent application, so it is favourable exactly when the ratio of applications to assemblies is high — Krylov iterations within a Newton step, repeated smoothing at a fixed state within a multigrid cycle — and unfavourable when the state changes every application. With break-even below 1.2, the threshold is essentially always met in a Newton–Krylov context.

Where it is inadmissible. The scheme perturbs the operator by $O(h)$. Used as the operator of record it changes the discrete problem. Its proper roles are as the Krylov operator inside an inexact-Newton iteration, where the residual is still evaluated exactly, and as a preconditioner. Quantifying the effect on Newton convergence for a specific forcing-term strategy is the most immediate open question and is not addressed here.

The remaining performance gap. At 18–19% of nominal vector peak the compute-bound elements are not close to the machine. Three causes are identifiable and separable: the gather and scatter at element boundaries, which the cost model charges as bandwidth but which also costs instructions and inhibits vectorisation at block edges; register spilling on HEX8, where Table 5 exceeds the register file; and the 1801 against 1558 operation gap to the hand-written HEX8 contraction, which is a scheduling question rather than a methodological one.

Element coverage and higher order. The two-dimensional kernels are generated and verified symbolically but have not been executed, pending support for the required affine geometry cache on triangular and quadrilateral meshes in the underlying mesh library. The rank grows with polynomial degree, so at sufficiently high order the factorisation ceases to pay — the trend is already visible at HEX8 in Table 3 — and sum factorisation becomes the appropriate technique instead. Determining the crossover degree is future work.

11 Conclusions

Projecting the pulled-back material tangent onto element-wise constants removes the quadrature loop from the matrix-free application of a nonlinear elasticity operator, leaving an application that reads a 45-number store and the increment and nothing else. The reference tensor that remains is a Gram matrix whose rank is the dimension of the span of the element’s gradient functions; it is factored exactly in rational arithmetic and folded into the emitted kernel as literal constants.

A per-element cost model places the resulting kernels at 1.0, 4.9 and 6.9 flop/byte for TET4, TET10 and HEX8, predicts that the first is limited by its own store and the other two by the contraction, and correctly orders the measured speed-ups over the exact matrix-free application ($2.4\times$, $8.1\times$, $2.8\times$). At 1.04×10^7 dof on eight cores the generated TET10 kernel is $1.60\times$ faster than a hand-optimised partial-assembly implementation of the same material, with a $1.67\times$ faster assembly, and HEX8 is at parity; strong scaling on 72 Arm cores is $69.4\times$ and $73.5\times$. Reduced-precision storage does not pay at the problem sizes measured, for reasons the cost model accounts for.

Three directions follow. The first is the inexact-Newton analysis: with an $O(h)$ Jacobian perturbation, what forcing-term strategy retains fast convergence, and how does the resulting iteration count trade against the per-application saving? The second is closing the gap to vector peak, where the gather–scatter and HEX8’s register pressure are the identified causes. The third is rank truncation: the modes are explicit, and discarding the smallest is a controlled second approximation.

Acknowledgements

To be completed.

References

- [1] A. Abdelfattah et al. A survey of numerical linear algebra methods utilizing mixed-precision arithmetic. *Int. J. High Perform. Comput. Appl.*, 35(4):344–369, 2021.
- [2] R. Anderson et al. MFEM: A modular finite element methods library. *Comput. Math. Appl.*, 81:42–74, 2021.
- [3] M. S. Alnæs, A. Logg, K. B. Ølgaard, M. E. Rognes, G. N. Wells. Unified Form Language: A domain-specific language for weak formulations of partial differential equations. *ACM Trans. Math. Softw.*, 40(2):9, 2014.
- [4] J. Bonet, R. D. Wood. *Nonlinear Continuum Mechanics for Finite Element Analysis*. Cambridge University Press, 2nd edition, 2008.
- [5] J. H. Bramble, S. R. Hilbert. Estimation of linear functionals on Sobolev spaces with application to Fourier transforms and spline interpolation. *SIAM J. Numer. Anal.*, 7(1):112–124, 1970.

- [6] J. Brown. Efficient nonlinear solvers for nodal high-order finite elements in 3D. *J. Sci. Comput.*, 45(1–3):48–63, 2010.
- [7] J. Brown et al. libCEED: Fast algebra for high-order element-based discretizations. *J. Open Source Softw.*, 6(63):2945, 2021.
- [8] P. G. Ciarlet. *The Finite Element Method for Elliptic Problems*. North-Holland, 1978.
- [9] R. S. Dembo, S. C. Eisenstat, T. Steihaug. Inexact Newton methods. *SIAM J. Numer. Anal.*, 19(2):400–408, 1982.
- [10] M. O. Deville, P. F. Fischer, E. H. Mund. *High-Order Methods for Incompressible Fluid Flow*. Cambridge University Press, 2002.
- [11] S. C. Eisenstat, H. F. Walker. Choosing the forcing terms in an inexact Newton method. *SIAM J. Sci. Comput.*, 17(1):16–32, 1996.
- [12] N. J. Higham, T. Mary. Mixed precision algorithms in numerical linear algebra. *Acta Numerica*, 31:347–414, 2022.
- [13] R. C. Kirby, A. Logg. A compiler for variational forms. *ACM Trans. Math. Softw.*, 32(3):417–444, 2006.
- [14] D. A. Knoll, D. E. Keyes. Jacobian-free Newton–Krylov methods: a survey of approaches and applications. *J. Comput. Phys.*, 193(2):357–397, 2004.
- [15] T. Kolev et al. Efficient exascale discretizations: High-order finite element methods. *Int. J. High Perform. Comput. Appl.*, 35(6):527–552, 2021.
- [16] M. Kronbichler, K. Kormann. A generic interface for parallel cell-based finite element operator application. *Comput. Fluids*, 63:135–147, 2012.
- [17] M. Kronbichler, K. Kormann. Fast matrix-free evaluation of discontinuous Galerkin finite element operators. *ACM Trans. Math. Softw.*, 45(3):29, 2019.
- [18] J. M. Melenk, K. Gerdes, C. Schwab. Fully discrete *hp*-finite elements: fast quadrature. *Comput. Methods Appl. Mech. Engrg.*, 190(32–33):4339–4364, 2001.
- [19] R. W. Ogden. Large deformation isotropic elasticity — on the correlation of theory and experiment for incompressible rubberlike solids. *Proc. R. Soc. Lond. A*, 326:565–584, 1972.
- [20] S. A. Orszag. Spectral methods for problems in complex geometries. *J. Comput. Phys.*, 37(1):70–92, 1980.
- [21] F. Rathgeber et al. Firedrake: Automating the finite element method by composing abstractions. *ACM Trans. Math. Softw.*, 43(3):24, 2016.